Question of the day

- How can we use our understanding of Big-Oh to show that the algorithm below always takes $\Theta(n^2)$ time?

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$data[1] \leq \ldots \leq data[n]$

```
for i = 1 to n − 1 do
    Let m be the location of the min value in data[i..n]
    Swap data[i] and data[m]
end
return data
```

Analysis

William Hendrix

Lecture 2

Today

- Review
  - Big-Oh notation
- Big-Oh properties
- Analyzing algorithms
- Summation identities
- Recursive analysis

Review: Big-Oh formal definitions

- Provides a useful way to classify functions according to growth rate
  - Focuses on asymptotic (eventual) growth
- O(*g(n)*): grow no faster than *g(n)*
- Ω(*g(n)*): grow no slower than *g(n)*
- Θ(*g(n)*): grow at the same rate as *g(n)*

$f(n) = O(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$f(n) = \Omega(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

- *To prove:* find positive constants that satisfy definition

Proof advice

- General proof strategy: find values of c and n_o
  - "Fill-in-the-blank" proof

$$f(n) \leq (\underset{\text{pos. const.}}{\underline{\qquad\qquad}})g(n) \quad \forall n \geq (\underset{\text{pos. const.}}{\underline{\qquad\qquad}})$$

- Finding smallest c or n_o isn't necessary
  - Choosing well can make your life easier, though
- Small powers of n are bounded above by larger powers
- Functions with negative coefficients are bounded above by zero
- Large coefficients are bounded above by n
  - With sufficiently large n_o

Big-Oh example (series)

- Prove that $\sum_{i=1}^{n} i = \Omega(n^2)$.

$1+2+3 \ldots + n = O(n^2)$

1 + 2 + 3 + ... + n >= (n/2) + ... + n
>= n/2 + n/2 + ... + n/2 = (n/2)(n/2)
>= (1/4) n^2
for all n >= (1)

Big-Oh example (series)

- Prove that $\sum_{i=1}^{n} i = \Omega(n^2)$.

Proof. Consider $\sum_{i=1}^{n} i$:

Good trick:
drop lower half
of series

$$
\begin{aligned}
\sum_{i=1}^{n} i &= 1 + 2 + \ldots + (n-1) + n \\
&\geq \underbrace{\lceil n/2 \rceil + \ldots + (n-1) + n}_{\geq n/2-1} \\
&\geq n/2(n/2-1) \\
&\geq n/2(n/4) && \forall n \geq 4 \\
&\geq n^2/8 && \forall n \geq 4
\end{aligned}
$$

Thus, there exist positive constants c and n_0, namely $c = 1/8$ and $n_0 = 2$, such that $\sum_{i=1}^{n} i \geq cn^2$ for all $n \geq n_0$, so $\sum_{i=1}^{n} i = \Omega(n^2)$. $\square$

Connection to calculus

- You can also determine O, Ω, and Θ by limits:

g grows faster $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \quad \to f(n) = O(g(n))$

Actually $f(n) = o(g(n))$

Same growth rate $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in (0, \infty) \to f(n) = \Theta(g(n))$

g grows slower $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \quad \to f(n) = \Omega(g(n))$

Actually $f(n) = \omega(g(n))$

- Standard rules for taking limits apply
  - Including L'Hôpital's Rule

Analysis of Big-Oh

Pros

- Provides a useful summary of the growth rate of the complexity
- Compact
- Simple: eight classes cover most useful algorithms

$$O(1) \ll O(\lg n) \ll O(n) \ll O(n \lg n) \ll O(n^2) \ll O(n^3) \ll O(2^n) \ll O(n!)$$

Cons

- Ignores contributions from coefficients and lower-order terms
- Doesn't rank algorithms with same growth rate
- Doesn't rank algorithms on small inputs
- Some of the "best" algorithms have extremely large coefficients, making them impractical for many purposes

Formal definition extra practice

- Use the *formal definition* of Big-Theta to prove:

  For any $x > 0$, if $f(n) = \Theta(g(n))$, then $xf(n) = \Theta(g(n))$

Formal definition extra practice

- Use the *formal definition* of Big-Theta to prove:

  For any $x > 0$, if $f(n) = \Theta(g(n))$, then $xf(n) = \Theta(g(n))$

  Proof. Since $f(n) = \Theta(g(n))$, there exist positive constants c and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$, for all $n \geq n_0$. Since $x > 0$, we may multiply all sides of this inequality by x, yielding $xc_1 g(n) \leq xf(n) \leq xc_2 g(n)$. Thus, there exist constants $c_3 = xc_1$, $c_4 = xc_2$, and n_0 such that $c_3 g(n) \leq xf(n) \leq c_4 g(n)$ for all $n \geq n_0$, so $xf(n) = \Theta(g(n))$. $\square$

Properties of Big-Oh notation

- Reflexivity

  $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, and $f(n) = \Theta(f(n))$

- Anti-symmetry

  $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

  $f(n) = O(g(n))$ and $f(n) = \Omega(g(n)) \Leftrightarrow f(n) = \Theta(g(n))$

- Symmetry (Θ only)

  $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

- Transitivity

  $f(n) = O(g(n))$ and $g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$

  $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \rightarrow f(n) = \Omega(h(n))$

  $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$

- Alternately:

  $O(O(h(n))) = O(h(n))$

  $\Omega(\Omega(h(n))) = \Omega(h(n))$

  $\Theta(\Theta(h(n))) = \Theta(h(n))$

$O(\Omega(h(n)))$

$\Omega(O(h(n)))$

Combination properties

- Envelopment
  - Addition

    $$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$
    $$\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n) + g(n))$$
    $$\Theta(f(n)) + \Theta(g(n)) = \Theta(f(n) + g(n))$$
  - Multiplication

    $$O(f(n))O(g(n)) = O(f(n)g(n))$$
    $$\Omega(f(n))\Omega(g(n)) = \Omega(f(n)g(n))$$
    $$\Theta(f(n))\Theta(g(n)) = \Theta(f(n)g(n))$$
- All three ignore constant coefficients

$$\forall x > 0, \quad \begin{aligned} f(n) = O(g(n)) &\rightarrow xf(n) = O(g(n)) \\ f(n) = \Omega(g(n)) &\rightarrow xf(n) = \Omega(g(n)) \\ f(n) = \Theta(g(n)) &\rightarrow xf(n) = \Theta(g(n)) \end{aligned}$$

- Only the largest term matters

$$f(n) = O(g(n)) \rightarrow O(f(n) + g(n)) = O(g(n))$$
$$f(n) = O(g(n)) \rightarrow \Omega(f(n) + g(n)) = \Omega(g(n))$$
$$f(n) = O(g(n)) \rightarrow \Theta(f(n) + g(n)) = \Theta(g(n))$$

Big-Oh properties example

- Use Big-Oh properties to establish the following:

1. If $f(n) = 13n^2 + 1234n + 91.2n\sqrt{n}$, then $f(n) = \Theta(n^2)$. Use the facts that $n = O(n^2)$ and $\sqrt{n} = O(n)$.

$n\sqrt{n} = O(n^2)$

```
f(n) = 13n^2 + 1234n + 91.2n sqrt(n)
= Theta(13n^2) + Theta(1234n) + Theta(91.2n sqrt(n))
= Theta(n^2) + Theta(n) + Theta(n sqrt(n))
= Theta(n^2 + n) + Theta(n sqrt(n))
= Theta(n^2) + Theta(n sqrt(n))
= Theta(n^2 + n sqrt(n))
= Theta(n * (n + sqrt(n))
= Theta(n) * Theta(n + sqrt(n))
= Theta(n) * Theta(n)
= Theta(n^2)
```

Big-Oh properties example

- Use Big-Oh properties to establish the following:

1. If $f(n) = 13n^2 + 1234n + 91.2n\sqrt{n}$, then $f(n) = \Theta(n^2)$. Use the facts that $n = O(n^2)$ and $\sqrt{n} = O(n)$.

$$
\begin{aligned}
f(n) &= 13n^2 + 1234n + 91.2n\sqrt{n} \\
&= \Theta(13n^2) + \Theta(1234n) + \Theta(91.2n\sqrt{n}) && \text{Reflexive} \\
&= \Theta(n^2) + \Theta(n) + \Theta(n\sqrt{n}) && \text{Constant coefficients} \\
&= \Theta(n^2 + n) + \Theta(n\sqrt{n}) && \text{Envelopment } (+) \\
&= \Theta(n^2) + \Theta(n\sqrt{n}) && \text{Largest term } (n = O(n^2)) \\
&= \Theta(n^2 + n\sqrt{n}) && \text{Envelopment } (+) \\
&= \Theta(n(n + \sqrt{n})) \\
&= \Theta(n)\Theta(n + \sqrt{n}) && \text{Envelopment } (\cdot) \\
&= \Theta(n)\Theta(n) && \text{Largest term } (\sqrt{n} = O(n)) \\
&= \Theta(n^2) && \text{Envelopment } (\cdot)
\end{aligned}
$$

Revenge of the logarithms

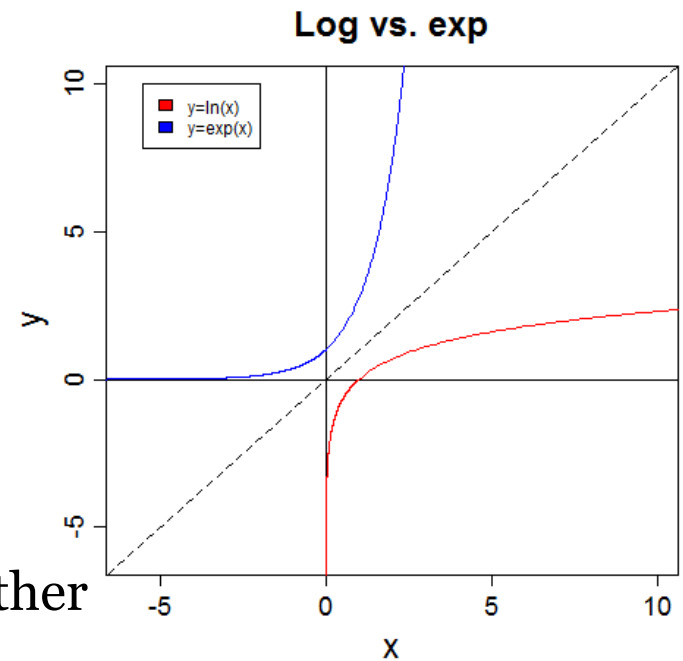
- **Logarithm:** inverse exponential function

$$y = \ln x \Leftrightarrow x = e^y$$

- Natural log (ln): inverse of e^x
- Logarithms of other bases: $\log_b(x)$
  - $\log_2(x)$ is very common in algorithms

lg

- Computing logs of other bases
  - $\log_b(x) = \frac{\ln x}{\ln b}$ $\frac{1}{\ln b}$ $\ln x$
  - All logs are *scalar multiples* of one another



- **Log properties**

Base 2 → $\lg(ab) = \lg(a) + \lg(b)$

$$\lg(a^b) = b\lg(a)$$

$$\boxed{\sum_{i=1}^{n} \frac{1}{i} = \Theta(\lg n)}$$

Because

$$2^A 2^B = 2^{A+B}$$

$$\left(2^A\right)^b = 2^{Ab}$$

$$\int_1^n \frac{1}{x} dx = \ln n$$

Logarithm property example

- Prove that $\lg(n!) = \Theta(n \lg(n))$.

lg(n!) = lg(n * (n-1) * (n-2) * ... * 3 * 2 * 1)
= lg n + lg(n-1) + lg(n-2) + ... + lg(3) + lg(2) + lg(1)
<= lg n + lg n + ... + lg n
= O(n lg n)

... = lg n + lg(n-1) + lg(n-2) + ... + lg(3) + lg(2) + lg(1)
>= lg n + lg(n-1) + ... lg(n/2)
>= lg(n/2) + lg(n/2) + ... + lg(n/2)
= 0.5n * lg(n/2)
= 0.5n * (lg n - 1)
= 0.5n * Omega(lg n)
= Omega(n lg n)
Goal: lg(n!) = Theta(n lg n)

Logarithm property example

- Prove that $\lg(n!) = \Theta(n \lg(n))$.

Proof. First, note that:

$$\begin{aligned} \lg(n!) &= \lg(n(n-1)(n-2)\cdots(2)(1)) && (1) \\ &= \lg(n) + \lg(n-1) + \ldots + \lg(2) + \lg(1) && (2) \end{aligned}$$

Since each term of this sum is less than or equal to $\lg(n)$,

$$\begin{aligned} \lg(n!) &\leq \lg(n) + \lg(n) + \ldots + \lg(n) + \lg(n) && (3) \\ &\leq n \lg(n) && (4) \end{aligned}$$

So, $\lg(n!) = O(n \lg(n))$. On the other hand, the sum in line 2 has at least $n/2$ elements that are $\lg(n/2)$ or larger, so:

$$\begin{aligned} \lg(n!) &\geq (n/2) \lg(n/2) && (5) \\ &\geq (n/2)(\lg n - \lg 2) && (6) \\ &\geq (n/2)(\lg n - 1) && (7) \\ &\geq (n/2) \lg n - n/2 && (8) \end{aligned}$$

Since $n/2 \lg n = \Omega(n \lg n)$ and $n/2 = \Omega(n)$, $(n/2) \lg n - n/2 = \Omega(n \lg n)$. Since $\lg(n!)$ bounded below by $\Omega(n \lg n)$, $\lg(n!) = \Omega(n \lg n)$. Therefore, $\lg(n!) = \Theta(n \lg n)$, because $\lg(n!) = O(n \lg n)$ and $\lg(n!) = \Omega(n \lg n)$. $\square$

Back to our previous example…

Input: $data$: array of integers
Input: n: size of $data$
Output: index min such that $data[min] \leq data[j]$, for all j from 1 to n

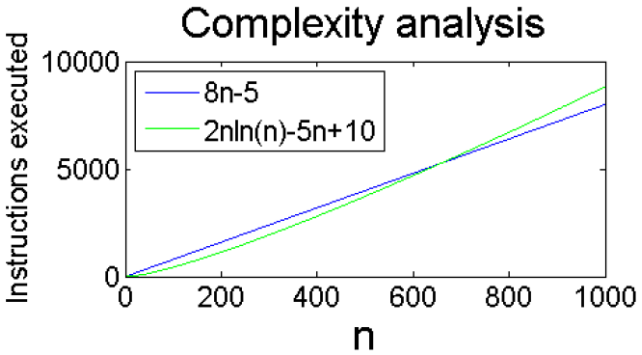
```
Algorithm: FindMin
min = 1
for i = 2 to n do
    if data[i] < data[min] then
        min = i
    end
end
return min
```

Ops per line	Times executed
…	…

Total ops: $\leq 7(n-1) + 2 = 7n - 5$

Question: is this better or worse than an algorithm that takes at most $2n \ln n - 5n + 10$ ops?

Better unless n < 395



Worst case: *O(n)*

Don't need to count instructions!

Other algorithm: *O(n*lg*(n))*

Conclusion: Our algorithm is better for sufficiently large *n*.

Algorithm analysis

- Identify loops and function calls
  - Everything else is *Θ(1)*
- *For loops*:
  - Estimate number of iterations
  - Estimate loop body running time
  - Does loop run time depend on iteration #?
    - If not: total = # iterations * time per iteration
    - Otherwise: total = sum of all iterations
- *For functions:* *
  - Non-recursive: analyze separately
  - Recursive: set up a recurrence and solve (after exam)
- Overall complexity: sum of complexities
  - Largest loop/function call

f( )

Summations

- Summations show up frequently in analysis

- Notation
  - "Dotty notation": $1 + 2 + 3 + \ldots + n$
    - Easy to understand
  - Compact notation: $\sum_{i=1}^{n} i$
    - Easy to write

- Useful identities

$$\sum_{i=1}^{x} i = \Theta(x^2) \qquad \sum_{i=1}^{x} 2^i = \Theta(2^x)$$

$$\sum_{i=1}^{x} \frac{1}{i} = \Theta(\lg x) \qquad \sum_{i=1}^{x} \frac{1}{2^i} = \Theta(1)$$

Loop analysis example

- What is the worst-case time complexity for the following algorithm?
  - What is the best-case?

Input: *data*: array of integers
Input: n: length of *data*
Output: permutation of *data* such that
$data[1] \leq data[2] \leq \ldots \leq data[n]$

```
Algorithm: SelectionSort
for i = 1 to n − 1 do
    min = i
    for j = i + 1 to n do
        if data[j] < data[min] then
            min = j
        end
    end
    Swap data[i] and data[min]
end
return data
```

Loop analysis example

- SelectionSort is $\Theta(n^2)$.

Proof. Note that the **for** loop in lines 2–10 will iterate $n - 1 = \Theta(n)$ times. Line 3 takes $\Theta(1)$ time, and line 9 takes $\Theta(1)$ (three assignment statements). The **for** loop in lines 4–8 iterates $n - i$ times. Lines 5 and 6 are both $\Theta(1)$, so each iteration of the **for** loop in lines 4–8 takes $\Theta(1)$ time. Since each iteration takes $\Theta(1)$ time and the loop iterates $n - i$ times, this **for** loop takes $\Theta(n - i)$ time. Hence, each iteration of the outer **for** loop takes $\Theta(1) + \Theta(n - i) + \Theta(1) = \Theta(n - i)$ time. Since the length of an iteration of the outer **for** loop depends on the iteration number, the total time will be $\sum_{i=1}^{n-1} \Theta(n - i) = \Theta\left(\sum_{i=1}^{n-1} i\right) = \Theta((n - 1) + (n - 2) + \ldots + 1)$ time. This sum equals $\Theta(\sum_{i=1}^{n-1} i)$, which is $\Theta(n^2)$. The **return** statement in line 11 takes $\Theta(1)$ time, so the total time for SelectionSort is $\Theta(n^2) + \Theta(1) = \Theta(n^2)$. $\square$

Analysis further practice

1. What is the *worst-case* time complexity for the algorithm below? Show your work.
   - *Best case?*

$\boxed{O(n^2)} = \boxed{\sum_{i=2}^{n} i}$

Input: *data*: an array of integers to sort
Input: n: the number of values in *data*
Output: permutation of *data* such that
$data[1] \leq \ldots \leq data[n]$

Algorithm: InsertionSort
for $i = 2$ to n **do**
  $ins = data[i]$
  $j = i$
  while $j > 1$ and $data[j-1] > ins$ **do**
    $data[j] = data[j-1]$
    $j = j - 1$
  end
  $data[j] = ins$
end
return *data*

$\Theta(n)$ iters @ $O(i)$ iters

$\Theta(1)$

$\Theta(1)$ iter

$O(i)$ iters @ $\boxed{\Theta(1)}$ iter

$O(1)$

$\Theta(1)$

Analysis further practice

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$$data[1] \leq \ldots \leq data[n]$$

```
Algorithm: InsertionSort
for i = 2 to n do                                  Θ(n) iter.
    ins = data[i]                                  Θ(1)
    j = i                                          Θ(1)
    while j > 1 and data[j − 1] > ins do           Θ(1)
        data[j] = data[j − 1]                      Θ(1)
        j = j − 1                                  Θ(1)
    end
    data[j] = ins                                  Θ(1)
end
return data
```

Analysis further practice

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$$data[1] \leq \ldots \leq data[n]$$

```
Algorithm: InsertionSort
for i = 2 to n do                                  Θ(n) iter.
    ins = data[i]                                  Θ(1)
    j = i
    while j > 1 and data[j − 1] > ins do           O(n), Ω(1) iter.
        data[j] = data[j − 1]                      Θ(1)
        j = j − 1
    end
    data[j] = ins                                  Θ(1)
end
return data
```

Analysis further practice

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$$data[1] \leq \ldots \leq data[n]$$

```
Algorithm: InsertionSort
for i = 2 to n do                                  Θ(n) iter.
    ins = data[i]                                  Θ(1)
    j = i                                          Θ(1)
    while j > 1 and data[j − 1] > ins do           O(n), Ω(1)
        data[j] = data[j − 1]                      O(n), Ω(1)
        j = j − 1                                  O(n), Ω(1)
    end                                            O(n), Ω(1)
    data[j] = ins                                  Θ(1)
end
return data
```

Analysis further practice

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$$data[1] \leq \ldots \leq data[n]$$

Algorithm: InsertionSort

$\Theta(n)$ iter @ $\Omega(1)$

$\Theta(n)(O(n), \Omega(1))$
$= O(n^2), \Omega(n)$

```
for i = 2 to n do
    ins = data[i]
    j = i
    while j > 1 and data[j − 1] > ins do
        data[j] = data[j − 1]
        j = j − 1
    end
    data[j] = ins
end
```

$\Omega(1)$

$\Theta(1)$

```
return data
```

Analysis further practice

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$data[1] \leq \ldots \leq data[n]$

Algorithm: InsertionSort

$O(n^2), \Omega(n)$

```
for i = 2 to n do
    ins = data[i]
    j = i
    while j > 1 and data[j − 1] > ins do
        data[j] = data[j − 1]
        j = j − 1
    end
    data[j] = ins
end
return data
```

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

Input: $data$: array of integers
Input: n: length of $data$
Output: permutation of $data$ such that
$data[1] \leq data[2] \leq \ldots \leq data[n]$

```
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

$\Theta(n)$ iters

$\Theta(\frac{n}{i})$ iters

$\Theta(n/i)$ iter

$\Theta(1)$ iter

$\Theta(1)$ iter

$n\left(1+\frac{1}{2}+\frac{1}{3}+\ldots+\frac{1}{n}\right) = n + \frac{n}{2} + \ldots + \frac{n}{n-1} = \sum_{i=1}^{n-1} n/i$

$= \Theta(n \lg n)$

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

$\frac{n-i}{i}$ iter.
$= \Theta(\frac{n}{i})$

```
Input: data: array of integers
Input: n: length of data
Output: permutation of data such that
        data[1] ≤ data[2] ≤ ... ≤ data[n]
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

Input: $data$: array of integers
Input: n: length of $data$
Output: permutation of $data$ such that
$data[1] \leq data[2] \leq \ldots \leq data[n]$

```
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

$\Theta(n/i)$ iter.

$\Theta(1)$

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

Input: $data$: array of integers
Input: n: length of $data$
Output: permutation of $data$ such that
$data[1] \leq data[2] \leq \ldots \leq data[n]$

```
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

$\Theta(n)$ iter.

$\Theta(n/i)$

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

```
Input: data: array of integers
Input: n: length of data
Output: permutation of data such that
        data[1] ≤ data[2] ≤ ... ≤ data[n]
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

$$\sum_{i=1}^{n-1} \Theta(n/i)$$

$$= \Theta\left(n \sum_{i=1}^{n-1} 1/i\right)$$

$$= \Theta(n \lg n)$$

Coming up

- Recursive analysis
- Master Theorem
- Data structures review

- **Recommended reading (today):** Section 11.1
  - *Practice problems:* R-11.1, C-11.1, C-11.3 (complexity only)
- **Recommended reading:** Sections 2.2, 1.4, and 2.1
  - *Practice problems:* R-2.3, R-2.2, R-2.6abc, C-2.12, C-2.19, A-2.1